

AGIVANT AI DELIVERY METHODOLOGY

BOLT

How to Use BOLT

Operating Playbook

Version 1.0

Agivant • Internal Use

Six stages. One structure. Every delivery lead finds what they need without re-learning the document.

Executive Summary

BOLT (AI-Native Delivery Operating System) is Agivant's operating framework for engagements where AI-agent execution — coding, testing, or ops — replaces ad hoc, heroics-driven delivery. Its six stages, S0 through S5, share one fixed format — Purpose, Entry Criteria, Inputs, Instructions, Outputs, Tools, Pitfalls — so anyone can find what they need in seconds without re-learning the method per project.

Lifecycle Overview

Six stages, applied in sequence. S0 is a gate that determines whether BOLT applies at all; S1–S5 form the delivery lifecycle.

S0	Classify the project GATE — BEFORE THE LIFECYCLE BEGINS	
		↓
S1	Define business intent BUSINESS INTENT	
		↓
S2	Write the solution spec SOLUTION SPECIFICATION	
		↓
S3	Execute via agentic delivery AGENTIC DELIVERY	
		↓
S4	Verify quality & security goals VERIFICATION	
		↓
S5	Feed outcomes into learning ops LEARNING OPERATIONS	

S0

Classify the project

GATE — BEFORE THE LIFECYCLE BEGINS

ENTRY CRITERIA

A new engagement or workstream has been scoped and assigned a delivery lead.

OUTPUTS

A logged classification decision. No further BOLT action if classified “not AI-native.”

PURPOSE

Confirm the project has an AI-native execution layer before applying BOLT. A project with no agentic execution has nothing for later stages to attach to.

INPUTS & OUTPUTS**INPUTS**

Not applicable — gate stage.

OUTPUTS / EXIT CRITERIA

– A logged classification decision. No further BOLT action if classified “not AI-native.”

PROCESS FLOW**1**

Ask: will any part of this project's execution (coding, testing, ops) be done by an AI agent rather than a human?

2

If yes — proceed to Stage 1. If no — use the standard (non-BOLT) delivery process; stop here.

3

Log the classification decision in the kickoff record (one line: “AI-native: yes/no”, with reasoning).

RECOMMENDED TOOLS

None specified at this stage.

⚠ COMMON PITFALLS

– Classifying a project “AI-native” for using AI-adjacent tools (e.g., a chatbot UI) without agentic execution underneath — the test is execution, not topic.

KEY TAKEAWAY

BOLT applies only where execution is agentic — classification is the gate, not a formality.

S1

Define business intent

BUSINESS INTENT

ENTRY CRITERIA

Project classified AI-native (Stage 0 complete).

OUTPUTS

Signed-off outcome statement + quantified success metrics (fits on one page).

PURPOSE

Translate the business ask into a structured outcome statement an agent or engineer can act on directly — not a narrative document for human interpretation.

INPUTS & OUTPUTS

INPUTS

- Stakeholder ask (verbal or written)
- Existing business case or ROI justification

OUTPUTS / EXIT CRITERIA

- Signed-off outcome statement + quantified success metrics (fits on one page).

PROCESS FLOW

- 1 Interview the stakeholder: what problem are we solving, and how will we know it's solved?
- 2 Draft a one-sentence outcome statement and 1–3 quantified success metrics.
- 3 Get stakeholder sign-off before Stage 2 — a structured brief, not a multi-page BRD.

RECOMMENDED TOOLS

Perplexity

Gemini Deep Research

Glean

⚠ COMMON PITFALLS

- Success metrics that are vague or unmeasurable instead of quantified.
- Skipping sign-off and proceeding on assumed scope — causes rework at Verification.

KEY TAKEAWAY

No stage proceeds on an unsigned, unquantified intent statement.

S2

Write the solution spec

SOLUTION SPECIFICATION

ENTRY CRITERIA

Business intent signed off (Stage 1 complete).

OUTPUTS

A structured, task-broken spec document ready for tool selection.

PURPOSE

Convert business intent into an agent-executable specification — architecture, schema, and task breakdown an AI agent can implement against directly.

INPUTS & OUTPUTS

INPUTS

- Outcome statement and success metrics from Stage 1
- Existing system architecture / integration points

OUTPUTS / EXIT CRITERIA

- A structured, task-broken spec document ready for tool selection.

PROCESS FLOW

- 1 Define technical scope: systems, data sources, and interfaces involved.
- 2 Write the spec as discrete, agent-sized tasks with input, output, and pass/fail conditions; state non-functional requirements explicitly.
- 3 Treat the spec as living — expect revisions per feature or per gap found during delivery.

RECOMMENDED TOOLS

Eraser AI

InfraSketch

Linear Agent

Jira PD AI

⚠ COMMON PITFALLS

- Writing prose a human would read instead of discrete agent-executable tasks.
- Omitting non-functional requirements (security, PII) — must be explicit, not assumed.

KEY TAKEAWAY

The spec is the contract the agent executes against — ambiguity here becomes rework later.

ENTRY CRITERIA

Solution spec approved (Stage 2 complete).

OUTPUTS

Implementation complete, each task passing its spec-defined condition, with a review log.

PURPOSE

Select the toolchain from the Build Environment catalog, then have agents build the implementation against the spec; engineers shift from writing code to specifying and reviewing it.

INPUTS & OUTPUTS

INPUTS

- Approved spec with task breakdown
- Build Environment tool catalog

OUTPUTS / EXIT CRITERIA

- Implementation complete, each task passing its spec-defined condition, with a review log.

PROCESS FLOW

1

Select the toolchain per task category (coding, orchestration, testing, ops) from the Build Environment catalog — by fit and complexity, not last project's default — and document the choice.

2

Feed the spec's task breakdown to the selected coding agent(s), one task at a time.

3

Engineer reviews each output against the task's pass/fail condition — a spec-conformance check, not a general review.

4

Escalate spec gaps back to Stage 2 rather than letting the agent improvise; confirm deployment ownership before delivery starts.

RECOMMENDED TOOLS

Cursor

GitHub Copilot

Claude Code

Antigravity

⚠ COMMON PITFALLS

- Letting an agent “fill gaps” in an ambiguous spec instead of escalating — passes review but misses intent.
- Reusing the previous project's toolchain without checking fit, or leaving the choice undocumented.

KEY TAKEAWAY

Review checks spec conformance, not general code quality — gaps go back to the spec, not the agent's judgment.

S4

Verify quality & security goals

VERIFICATION

ENTRY CRITERIA

Agentic delivery complete (Stage 3 complete).

OUTPUTS

Test results against defined quality goals + a triaged defect log. No blocking defects open.

PURPOSE

Confirm the implementation meets the quality bar and security requirements the human defined — agents run the tests, a human sets the bar.

INPUTS & OUTPUTS

INPUTS

- Implementation from Stage 3
- Quality goals and security requirements from spec

OUTPUTS / EXIT CRITERIA

- Test results against defined quality goals + a triaged defect log. No blocking defects open.

PROCESS FLOW

- 1 Define quality goals explicitly if not already in the spec (accuracy, edge cases, response time).
- 2 Generate test cases against those goals, including edge/adversarial input; run a security review pass.
- 3 Log defects and triage severity (blocking vs. non-blocking) before closing verification.

RECOMMENDED TOOLS

Qase AIDEN

QAM Hub

PractiTest SmartFox

Antigravity

Mabl

Functionize

Sonar AI

⚠ COMMON PITFALLS

- Treating “tests pass” as sufficient without a defined bar.
- Skipping the security pass because functional tests passed.

KEY TAKEAWAY

“Tests pass” is not the exit bar — the defined quality goal and security pass are.

ENTRY CRITERIA

Verification complete with no blocking defects (Stage 4 complete).

OUTPUTS

A tagged, findable reusable asset entry on SharePoint, to migrate to the Learning Ops library once built.

PURPOSE

Capture what worked, what broke, and what pattern emerged — so the next similar project starts from this asset, not from zero.

INPUTS & OUTPUTS

INPUTS

- Defect log from Stage 4
- Final spec and toolchain used

OUTPUTS / EXIT CRITERIA

- A tagged, findable reusable asset entry on SharePoint, to migrate to the Learning Ops library once built.

PROCESS FLOW

- 1 Document the pattern used as a reusable asset — not just a project retrospective.
- 2 Log recurring defect types and root cause, distinct from one-off bugs.
- 3 Store the tagged asset on SharePoint (interim, until the Learning Ops library is built); flag adjustments for next iteration.

RECOMMENDED TOOLS

Glean

CloudBees Smart Tests

Notion AI

SharePoint

⚠ COMMON PITFALLS

- Writing a retrospective only the current team will read, instead of a tagged reusable pattern.
- Not distinguishing recurring defect classes from one-off bugs — loses the learning signal.

KEY TAKEAWAY

The exit artifact is a tagged reusable pattern, not a project-only retrospective.

Closing Principles

Four principles recur across BOLT stages. They are not additional steps — they are how the six stages are meant to be interpreted when a situation isn't explicitly covered.

ESCALATE GAPS, DON'T IMPROVISE AROUND THEM

S2 requires non-functional requirements to be explicit rather than assumed; S3 requires spec gaps to route back to Stage 2 rather than letting an agent fill them. The pattern is the same: ambiguity is a signal to escalate to the stage that owns the decision, not a gap for the next stage to absorb.

DECISIONS ARE DOCUMENTED, NOT INHERITED

S3 requires the toolchain to be selected on fit and documented before delivery starts — explicitly not carried over from the last project. The same discipline underlies S0's logged classification and S1's signed-off outcome statement: each is a recorded decision, not a default.

THE EXIT BAR IS DEFINED, NOT ASSUMED

S4 treats “tests pass” as insufficient without a defined quality and security bar set in advance. This mirrors S1's requirement for quantified, not vague, success metrics — BOLT consistently asks for the bar to be set before it is checked against.

EVERY ENGAGEMENT FEEDS THE NEXT ONE

S5 closes the lifecycle by requiring a tagged, reusable asset — not a project-only retrospective — so the next similar engagement starts from a pattern rather than from zero.